RutaSmart: Diseño, Implementación y Evaluación de un Asistente Inteligente para la Selección de Alternativas de Transporte en Bogotá

Sarah Isabella Peña Cortes
Facultad de Ingeniería, Universidad de San Buenaventura, Bogotá, Colombia
sipenac@academia.usbbog.edu.co
Optativa II: Desarrollo de Aplicaciones para Dispositivos Móviles · Docente: Ing. Jairo Armando Salcedo Aranda · Noviembre de 2026

***Resumen*—Este artículo presenta el diseño, la implementación y la evaluación de RutaSmart, una aplicación móvil nativa para Android desarrollada en Kotlin con Jetpack Compose que compara alternativas de transporte urbano (caminata, bicicleta, taxi, TransMilenio, SITP y combinaciones) y recomienda la más adecuada según prioridades ponderadas por el usuario: tiempo, costo, comodidad, transbordos e impacto ambiental. La red de transporte se modela como un grafo de 50 paradas y 10 líneas sobre el que se ejecuta una variante del algoritmo de Dijkstra; un motor de puntaje propio asigna a cada alternativa un valor de 0 a 100. Un asistente híbrido combina un motor local con el modelo Gemini de Google para explicar la recomendación en lenguaje natural sin inventar cifras. La aplicación usa arquitectura MVVM por capas, Room para el funcionamiento sin conexión, Firebase Authentication y Cloud Firestore para la sincronización, y un panel de administración protegido por reglas de seguridad para suspender líneas y publicar avisos. La lógica de dominio quedó cubierta por 80 pruebas unitarias aprobadas; el cálculo de alternativas tomó en promedio 0,34 ms por consulta en una JVM de escritorio, y el análisis estático de seguridad no reportó hallazgos de severidad alta ni media.**

***Palabras clave*—Kotlin, Android, Jetpack Compose, movilidad urbana, algoritmo de Dijkstra, inteligencia artificial, Firebase, MVVM, programación orientada a objetos.**

I. INTRODUCCIÓN

En Bogotá una persona puede llegar a su destino caminando, en bicicleta, en bus del SITP, en TransMilenio, en taxi o combinando medios. Cada opción difiere en tiempo, costo, transbordos, comodidad y emisiones, y esas diferencias no siempre son evidentes al momento de decidir. Las aplicaciones de navegación más usadas calculan rutas, pero su criterio de recomendación no es configurable por el usuario ni explican por qué una opción es mejor que otra.

RutaSmart responde a la pregunta: *¿cómo puede una aplicación móvil en Kotlin ayudar a una persona a seleccionar, de forma objetiva y personalizada, la alternativa de transporte más adecuada para llegar a un destino?* El documento de propuesta del proyecto planteó el diseño; este artículo reporta la solución construida, las decisiones que cambiaron frente a la propuesta y los resultados de las pruebas.

El resto del artículo se organiza así: la sección II describe la arquitectura; la III, el modelo de dominio y los algoritmos; la IV, el asistente de IA; la V, la persistencia y la seguridad; la VI, la metodología; la VII, las pruebas y resultados; y la VIII, las conclusiones.

II. ARQUITECTURA DEL SISTEMA

La aplicación sigue el patrón MVVM (Modelo–Vista–VistaModelo) organizado en tres capas, de acuerdo con la guía de arquitectura de Android [2] (Fig. 1).

A. Capa de presentación: ocho pantallas en Jetpack Compose con Material Design 3 (inicio de sesión, inicio, preferencias, alternativas, detalle, historial, asistente y administración). Los ViewModels exponen el estado con *StateFlow* y la navegación usa Navigation Compose con un botón flotante que abre el asistente desde cualquier pantalla.

B. Capa de dominio: Kotlin puro, sin dependencias de Android, con el modelo, el motor de rutas, los casos de uso, el núcleo del agente de IA y el módulo de administración. Esta independencia permite probarla con JUnit y moverla a un módulo de Kotlin Multiplatform.

C. Capa de datos: patrón *Repository* sobre Room (SQLite), Firebase (Authentication y Cloud Firestore) y el cliente HTTPS de Gemini. Las dependencias se crean una sola vez en un contenedor (inyección manual).

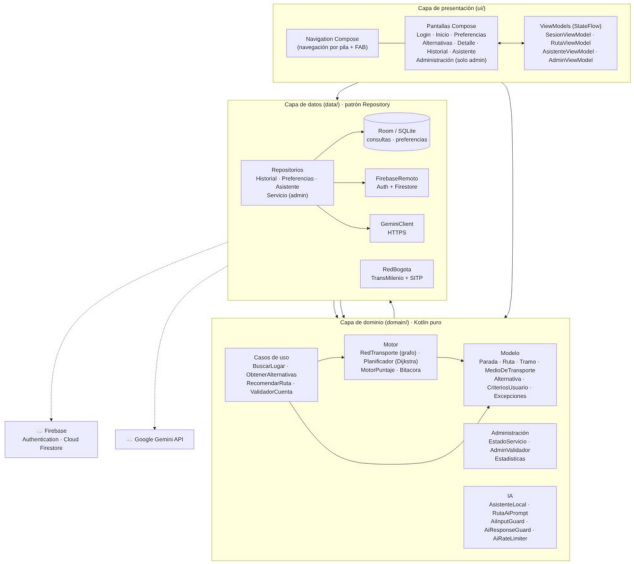


Fig. 1. Arquitectura por capas de RutaSmart (presentación, dominio y datos) y servicios externos.

TABLA I

TECNOLOGÍAS DE LA SOLUCIÓN FINAL

Componente	Tecnología	Uso
Lenguaje	Kotlin 1.9.24	100 % del código
Interfaz	Jetpack Compose · Material 3	8 pantallas, tema claro/oscuro
Persistencia local	Room 2.6.1 (SQLite)	Historial y preferencias
Nube	Firebase Auth + Firestore	Cuentas, sincronización, admin
Concurrencia	Coroutines + Flow	Cálculo y red fuera del hilo UI
IA	Google Gemini (API REST)	Explicación en lenguaje natural
Pruebas	JUnit 4	80 pruebas del dominio
Integración continua	Azure Pipelines (YAML)	Lint, pruebas y APK

Frente a la propuesta hubo tres cambios deliberados: (1) se reemplazó el SDK de Google Maps y la geolocalización por un mapa esquemático propio y un enlace a Google Maps, para no pedir el permiso de ubicación; (2) el modelo de IA es Gemini, que ofrece un nivel gratuito adecuado para un proyecto académico; y (3) no se usó Retrofit, porque la red de transporte es local y la única API REST (Gemini) se consume con un cliente HTTPS mínimo.

III. MODELO DE DOMINIO Y ALGORITMOS

A. Programación orientada a objetos

MedioDeTransporte es una interfaz implementada por cinco clases (Caminata, Bicicleta, Bus, Metro y Taxi); cada una define su propia velocidad, comodidad, emisiones y tarifa, de modo que el motor calcula duraciones y costos por **polimorfismo**. *Parada*, *Ruta*, *Tramo* y *Alternativa* encapsulan sus datos como propiedades de solo lectura, y las cuatro excepciones propias heredan de una clase sellada *RutaSmartException* que siempre trae un mensaje comprensible para el usuario.

B. Estructuras de datos, arreglos y ciclos

La red de ejemplo de Bogotá (4 troncales de TransMilenio y 6 rutas del SITP) se carga desde un arreglo de 50 paradas a un **grafo** representado con un *HashMap* de paradas y una lista de adyacencia de conexiones. Los ciclos *for* crean las conexiones entre paradas consecutivas; un ciclo *while* sincroniza el historial pendiente mientras haya elementos **y** conexión; y las estadísticas del panel de administración cuentan frecuencias con un *HashMap*.

C. Búsqueda de rutas (Dijkstra)

Para el transporte público se ejecuta el algoritmo de Dijkstra [1] con una cola de prioridad, cuyo estado es (parada, línea actual, ya abordó). Guardar la línea permite sumar la espera y la penalidad de transbordo solo al cambiar de vehículo, y la bandera «ya abordó» evita devolver como transporte público un camino que es solo caminata. Las distancias se calculan con la fórmula de Haversine [9]. Se buscan tres variantes (solo TransMilenio, solo SITP y TransMilenio + SITP) y se agregan las alternativas directas permitidas (caminata hasta 3,5 km, bicicleta hasta 12 km y taxi). Las líneas suspendidas por el administrador se excluyen del grafo.

D. Motor de puntaje

Para cada criterio con peso *w* (de 0 a 5), la alternativa recibe *(mejor + margen)/(valor + margen)*, donde *mejor* es el menor valor del grupo; la comodidad usa *valor/mejor*. El puntaje final es la media ponderada multiplicada por 100. Se probó primero una normalización mínimo–máximo, pero hacía ganar al taxi por pocos minutos aunque costara diez veces más; la normalización por proporción con márgenes corrigió ese sesgo (defecto D-02, Tabla IV).

IV. ASISTENTE DE INTELIGENCIA ARTIFICIAL

El asistente «Rutina» es **híbrido**: un motor local calcula la respuesta exacta (la opción más económica, la más rápida, el efecto de la hora pico, etc.) y Gemini [5] la redacta en lenguaje natural recibiendo el ranking real, los criterios y los tramos. Así la IA no puede inventar tiempos ni tarifas. Si no hay clave, no hay internet, se supera el límite o la respuesta es inválida, se muestra la respuesta local con el motivo.

Los controles del agente incluyen: validación de entrada de 1 a 500 caracteres y detección de intentos de manipulación de instrucciones; límite de 10 mensajes por minuto; limpieza y validación de la respuesta del modelo; traducción de los errores HTTP a mensajes comprensibles; y la clave fuera del código (*local.properties → BuildConfig*, enviada en un encabezado HTTPS). Cada respuesta se etiqueta como «IA» o «local» y la pantalla informa que el usuario conversa con un sistema de IA.

V. PERSISTENCIA, NUBE Y SEGURIDAD

Cada búsqueda se guarda primero en Room, de modo que la aplicación funciona sin conexión, y se sincroniza después con Cloud Firestore en *usuarios/{uid}/consultas*. Las reglas de seguridad de Firestore [4] permiten a cada usuario leer y escribir solo sus documentos y validan tipos y rangos (por ejemplo, puntaje entre 0 y 100).

El **panel de administración** se habilita al iniciar sesión con la cuenta administradora. Permite ver estadísticas anónimas de uso, suspender o reactivar líneas (el planificador deja de usarlas para todos los usuarios) y publicar avisos de hasta 200 caracteres sin enlaces. El rol se verifica dos veces: en la aplicación, para mostrar el menú, y en las reglas de Firestore, que rechazan cualquier escritura en *config/servicio* o lectura del grupo *consultas* que no provenga del administrador.

Siguiendo OWASP MASVS [6] se deshabilitaron los respaldos, se prohibió el tráfico HTTP sin cifrar, se protegió la actividad contra el secuestro de tareas, se limitaron los permisos a INTERNET y ACCESS_NETWORK_STATE y el registro en Logcat quedó solo para compilaciones de depuración.

VI. METODOLOGÍA

El proyecto se gestionó con Scrum [10] en tres sprints de dos semanas: (1) diseño (requisitos IEEE 830/29148 [7], mockups, arquitectura y clases); (2) dominio y motor

(modelo, grafo, Dijkstra, puntaje, excepciones y pruebas unitarias); y (3) aplicación, nube e IA (pantallas, Room, Firebase, asistente, administración, seguridad y documentación). Se definieron 21 requisitos funcionales, 10 no funcionales, 15 historias de usuario, 16 casos de uso y 45 casos de prueba, enlazados en una matriz de trazabilidad. Un pipeline YAML de Azure ejecuta lint, pruebas y la compilación del APK en cada cambio.

VII. PRUEBAS Y RESULTADOS

A. Pruebas unitarias

Las pruebas se diseñaron según IEEE 829 [8]. Las 80 pruebas JUnit del dominio se aprobaron en su totalidad (Tabla II); 28 de los 45 casos de prueba del plan quedan cubiertos por pruebas automáticas y los 17 restantes son casos manuales de interfaz y nube que se ejecutan en el emulador.

TABLA II

RESULTADO DE LAS PRUEBAS UNITARIAS

Clase de prueba	Pruebas	Resultado
ModeloTest	16	16/16
MotorTest	23	23/23
AsistenteYDatosTest	13	13/13
AiCoreTest	13	13/13
AdminTest	15	15/15
Total	80	80/80

B. Recomendaciones obtenidas

La Tabla III muestra la alternativa recomendada por el motor para seis trayectos reales de la red, con los criterios por defecto (tiempo 5, costo 3, comodidad 3, transbordos 2, ambiente 1, sin hora pico). Para trayectos medianos gana la bicicleta, por costo cero y emisiones nulas; para el trayecto largo hacia el centro, la combinación TransMilenio + SITP; y cuando el transporte público no conecta bien, el taxi.

TABLA III

ALTERNATIVA RECOMENDADA POR TRAYECTO

Trayecto	Recomendada	Min	Costo	Pts
U. San Buenaventura → Pl. de Bolívar	TM + SITP	73	$3.400	78,4
Unicentro → U. Nacional	Bicicleta	48	$0	77,9
Portal 80 → Zona T	Bicicleta	44	$0	78,5
Aeropuerto → C. Internacional	Taxi	48	$27.500	78,5
Portal Américas → Parque 93	Taxi	54	$30.600	78,5
Titán Plaza → Gran Estación	Bicicleta	31	$0	80,2

C. Rendimiento

Se midieron 300 cálculos completos (generación de alternativas, Dijkstra y puntaje) sobre los seis trayectos de la Tabla III, después de 180 ejecuciones de calentamiento, en una JVM de escritorio (Intel Xeon 2,3 GHz, 2 núcleos). El tiempo promedio fue de **0,34 ms**, la mediana de 0,25 ms, el percentil 95 de 0,76 ms y el máximo de 4,62 ms, muy por debajo del objetivo de 1 s del requisito RNF-03. En un teléfono el tiempo absoluto será mayor; su medición con el Profiler de Android Studio queda dentro de las pruebas en emulador.

D. Seguridad y defectos

El análisis estático con mobsfscan no reportó hallazgos de severidad alta ni media, y detect-secrets no encontró credenciales en el repositorio. Durante las pruebas se corrigieron cinco defectos (Tabla IV), cada uno con su re-prueba.

TABLA IV

DEFECTOS CORREGIDOS DURANTE LAS PRUEBAS

ID	Defecto	Corrección
D-01	Rutas de solo caminata como transporte público	Estado «ya abordó» en Dijkstra
D-02	El taxi ganaba aunque costaba 10× más	Normalización por proporción
D-03	Bicicleta recomendada para 15 km	Límite de 12 km y comodidad 2
D-04	Pasos confusos con nombres repetidos	Paradas «(paradero SITP)»
D-05	Log activo en producción	Log solo con BuildConfig.DEBUG

VIII. CONCLUSIONES

RutaSmart demuestra que una aplicación en Kotlin puede convertir una decisión cotidiana y subjetiva en una comparación objetiva y personalizada. Modelar la red como grafo y separar el dominio del resto de la aplicación permitió probar a fondo la lógica (80 pruebas) y obtener tiempos de cálculo de fracciones de milisegundo. El enfoque híbrido del asistente resolvió el principal riesgo de usar un modelo de lenguaje: las cifras siempre provienen del motor y la IA solo las explica.

Dificultades. Las principales fueron calibrar el puntaje para que una alternativa no ganara solo por un criterio, evitar caminos degenerados en Dijkstra y diseñar un rol de administrador que no dependiera únicamente de la aplicación, lo que se resolvió con reglas de Firestore.

Mejoras. (1) Reemplazar la red de ejemplo por los datos oficiales GTFS de TransMilenio; (2) incorporar la ubicación del usuario y horarios en tiempo real; (3) mover la clave de IA a un backend propio; y (4) compartir el dominio con iOS mediante Kotlin Multiplatform.

REFERENCIAS

[1] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[2] Google, "Guide to app architecture," *Android Developers*. [En línea]. Disponible: https://developer.android.com/topic/architecture

[3] JetBrains, "Null safety," *Kotlin Documentation*. [En línea]. Disponible: https://kotlinlang.org/docs/null-safety.html

[4] Google, "Get started with Cloud Firestore Security Rules," *Firebase Documentation*. [En línea]. Disponible: https://firebase.google.com/docs/firestore/security/get-started

[5] Google, "Gemini API documentation," *Google AI for Developers*. [En línea]. Disponible: https://ai.google.dev/gemini-api/docs

[6] OWASP Foundation, *Mobile Application Security Verification Standard (MASVS)*, v2.0, 2023.

[7] *Systems and software engineering — Life cycle processes — Requirements engineering*, ISO/IEC/IEEE 29148:2018.

[8] *IEEE Standard for Software and System Test Documentation*, IEEE Std 829-2008.

[9] R. W. Sinnott, "Virtues of the Haversine," *Sky and Telescope*, vol. 68, no. 2, p. 159, 1984.

[10] K. Schwaber y J. Sutherland, *La Guía de Scrum*, 2020. [En línea]. Disponible: https://scrumguides.org